

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	SHAFFRIA.A	Reg. No.:	192571047
Section / Batch:	B TECH CSBS	Date:	28.07.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I SHAFFRIA.A (192571047) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex realworld problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & DecisionMaking Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				

Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q75. A malware detection tool scans system logs using brute-force string matching to find suspicious patterns.

- Explain how the pattern matching algorithm works.
- Analyze its time complexity.
- Discuss inefficiency for large log files.

Q76. An online retail system searches for customer orders stored in an unsorted list using Linear Search.

- Explain how Linear Search is applied.
- Analyze performance degradation as data grows.
- Suggest improvements and justify.

```
main.py
1 def brute_force_string_match(text, pattern):
2     """
3     Performs brute-force string matching to find all occurrences
4     of a suspicious pattern in a system log text.
5     """
6     n = len(text)
7     m = len(pattern)
8     matches = []
9
10    # Slide pattern over text one by one
11    for i in range(n - m + 1):
12        j = 0
13        while j < m and text[i + j] == pattern[j]:
14            j += 1
15
16        # If pattern matches completely
17        if j == m:
18            matches.append(i)
19
20    return matches
21 system_log = "System warning: unauthorized access detected from IP 192.168.1.50. Malicious signature"
22 suspicious_pattern = "Malicious"
23
24 found_indices = brute_force_string_match(system_log, suspicious_pattern)
25 print(f"Pattern found at indices: {found_indices}")

```

input

Pattern found at indices: [67]

...Program finished with exit code 0
Press ENTER to exit console.

```
File
main.py (Ctrl+M)
1 def linear_search_orders(orders, target_order_id):
2     """
3     Searches for a customer order ID in an unsorted list using Linear Search.
4     """
5     for index, order in enumerate(orders):
6         if order["order_id"] == target_order_id:
7             return index, order # Return index and order details
8     return -1, None
9
10 # Example Execution with an Unsorted List of Orders
11 customer_orders = [
12     {"order_id": 1042, "item": "Laptop"},
13     {"order_id": 1011, "item": "Smartphone"},
14     {"order_id": 1085, "item": "Headphones"},
15     {"order_id": 1023, "item": "Keyboard"}
16 ]
17
18 search_target = 1085
19 idx, result = linear_search_orders(customer_orders, search_target)
20
21 if idx != -1:
22     print(f"Order found at index {idx}: {result}")
23 else:
24     print("Order not found.")

```

input

Order found at index 2: {'order_id': 1085, 'item': 'Headphones'}

...Program finished with exit code 0
Press ENTER to exit console.

Q75. Malware Detection Tool – Brute-Force String Matching

a) How the Pattern Matching Algorithm Works

- **Mechanism:** The brute-force string matching algorithm searches for a pattern of length m within a text (system log) of length n by aligning the pattern at every possible index of the text.
- **Process:** It starts at index 0 of the text and compares characters of the pattern one by one. If a mismatch occurs, the pattern is shifted one position to the right, and the comparison restarts.

b) Time Complexity Analysis

- **Worst-Case Time Complexity:** $O(n \times m)$, where n is the length of the system log and m is the length of the suspicious pattern. This happens when repeated character checks occur before a mismatch (e.g., text = AAAAAAAB, pattern = AAAB).
- **Best-Case Time Complexity:** $O(n)$, when the first character of the pattern does not match the text character, allowing the algorithm to slide across the text quickly.

c) Inefficiency for Large Log Files

- For massive system log files, checking every single offset redundantly results in excessive CPU overhead and latency.
- Advanced algorithms like KMP (Knuth-Morris-Pratt) or Boyer-Moore use preprocessing to skip unnecessary

shifts, which performs significantly better on large files.

Q76. Online Retail System – Linear Search in Unsorted List

a) How Linear Search is Applied

- **Mechanism:** Linear search scans through a list of customer orders sequentially, checking each element one by one from the beginning until the target order ID is found or the list ends.
- **Application:** Because the customer orders are stored in an unsorted list, there is no structural ordering (like binary search requires), meaning every single element has an equal chance of being inspected.

b) Analyze Performance Degradation as Data Grows

- **Time Complexity:** $O(n)$, meaning execution time grows linearly with the number of orders n .
- **Degradation Impact:** As an e-commerce platform expands to millions of customer orders, the search time increases drastically, leading to high latency, slower UI response times, and poor database/system throughput.

c) Suggest Improvements and Justification

- **Improvement:** Implement a **Hash Table (Dictionary)** or **Indexing/Sorting with Binary Search**.
- **Justification:**
 - Using a Hash Table allows order lookups in $O(1)$ average time complexity because order keys map

directly to memory locations, eliminating sequential checks entirely.

- Sorting the orders or maintaining a balanced Search Tree (like a B-Tree used in databases) reduces search time to $O(\log n)$, scaling efficiently as data grows.
